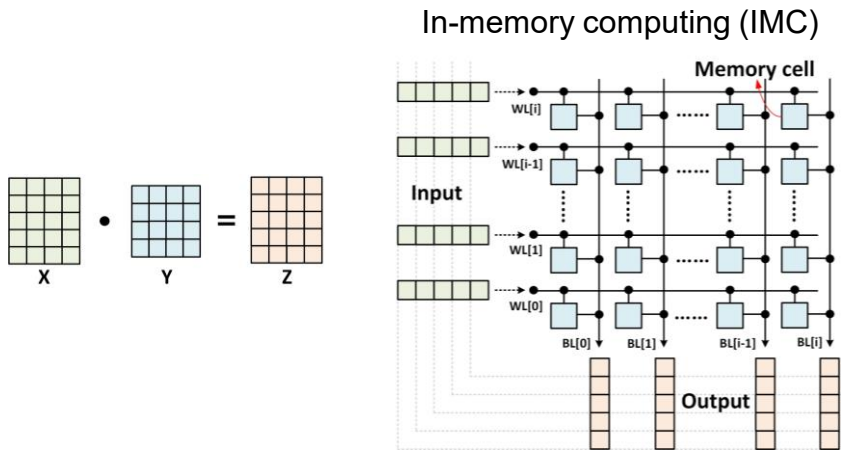


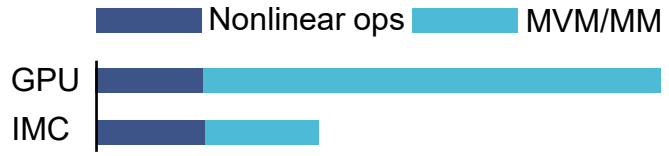
OmniAcc: A Heterogeneous Memory-Centric LLM Accelerator with Reconfigurable Fixed-Point Nonlinear Operations and Layer-Agnostic Compute Fabric

Yimin Wang*, Zixuan Wang*, Yue Jiet Chong, Xuanyao Fong
National University of Singapore

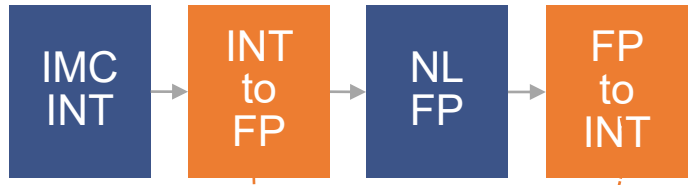
Why Accelerating Matrix Multiplication Is Not Enough

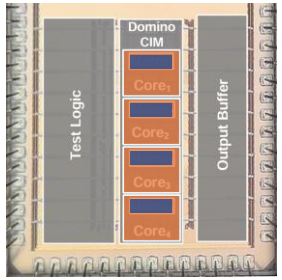
Nonlinear processing overhead





Precision and conversion overhead







Source: Li et al. (2026)

Accelerating linear operations exposes nonlinear processing and precision conversion as bottlenecks

OmniAcc: Three Connected Design Challenges

RQ 1 | Can nonlinear functions use fixed-point arithmetic without material accuracy loss? — Algorithm

Solution 1 | Outlier-aware, accuracy-preserving fixed-point quantization

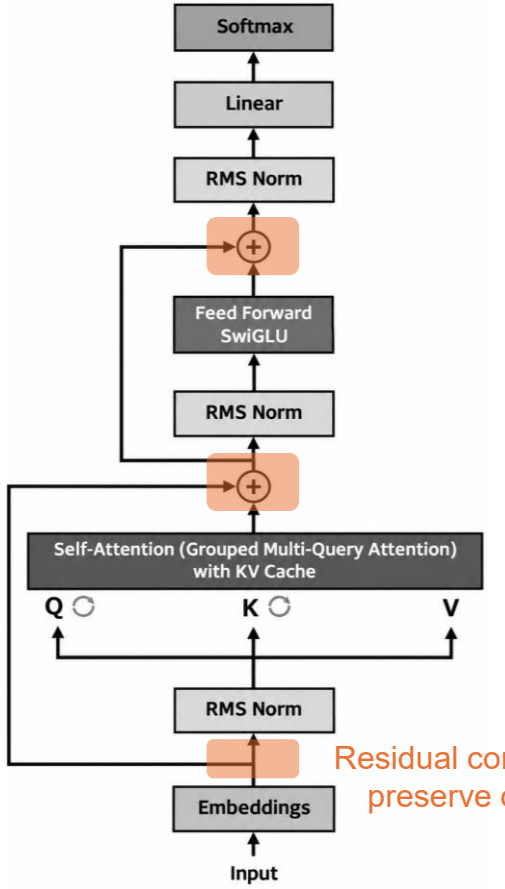
RQ 2 | How can nonlinear processing be integrated efficiently into a scalable architecture? — Architecture

Solution 2 | Reconfigurable nonlinear units and regularly interleaved functional slices

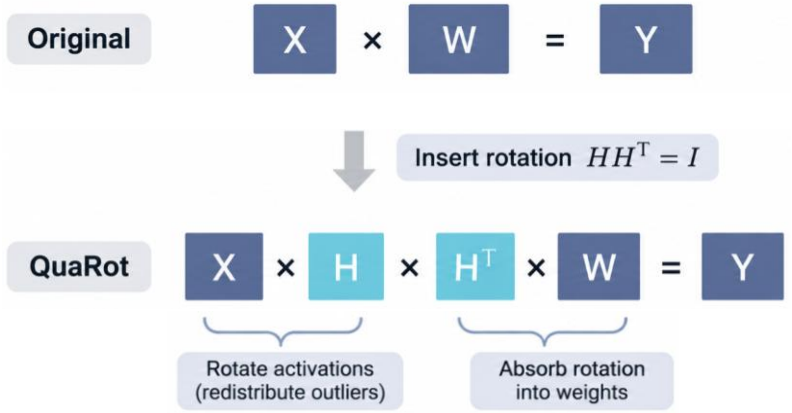
RQ 3 | How can computation and communication be orchestrated with low runtime overhead? — System

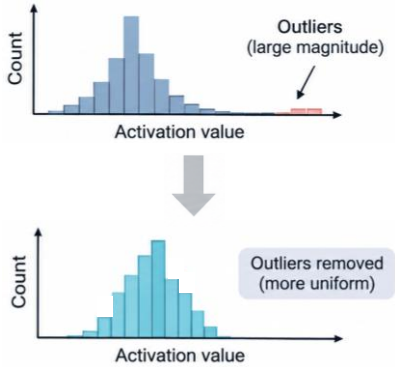
Solution 3 | Stationarity-aware mapping with predictable, deterministic dataflow

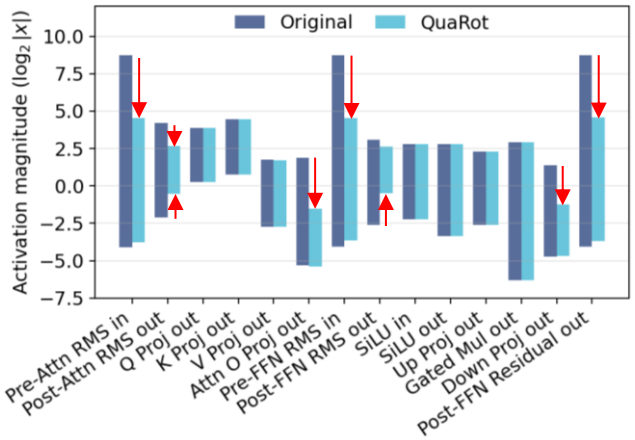
Outlier-Aware Rotation Makes Fixed-Point Processing Feasible



Residual connections preserve outliers







Walsh–Hadamard transform to distribute outliers across channels

No runtime weight-conversion overhead

← Reduced activation range in Llama models

Outlier redistribution reduces the activation range required for fixed-point processing

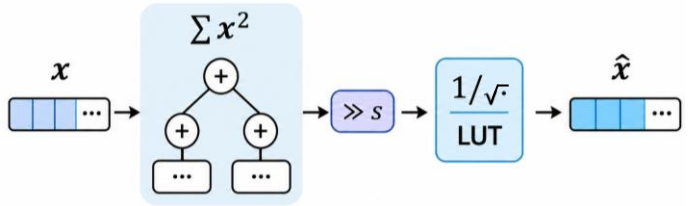
Operation-Specific Approximations Preserve Model Quality

Fixed-point nonlinear processing

RMSNorm

Hierarchical reduction with shift-based scaling

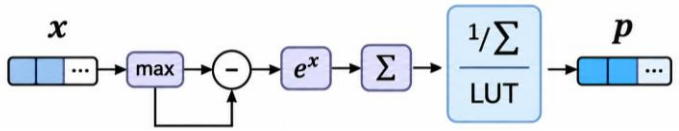
32-entry reciprocal-square-root lookup table



Softmax

Max subtraction and thresholding below −12

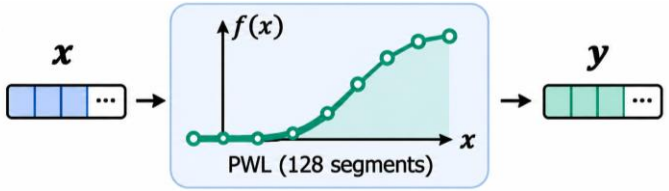
PWL exponential + reciprocal lookup tables



SiLU

128-segment piecewise-linear approximation

Elementwise activation without explicit division



Combined nonlinear approximation

Llama 3.2-1B

WikiText-2 perplexity

9.9893 → 10.0114

Average zero-shot accuracy

60.99% → 61.22%

Llama 3.1-8B

WikiText-2 perplexity

6.4649 → 6.4826

Average zero-shot accuracy

69.66% → 69.42%

Function-specific nonlinear approximations preserve near-baseline model quality

Next: Architecture and Hardware Design

RQ 1 | Can nonlinear functions use fixed-point arithmetic without material accuracy loss? — Algorithm

Solution 1 | Outlier-aware, accuracy-preserving fixed-point quantization

RQ 2 | How can nonlinear processing be integrated efficiently into a scalable architecture? — Architecture

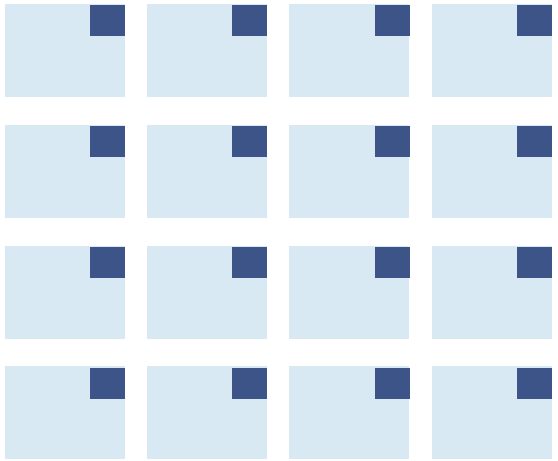
Solution 2 | Reconfigurable nonlinear units and regularly interleaved functional slices

RQ 3 | How can computation and communication be orchestrated with low runtime overhead? — System

Solution 3 | Stationarity-aware mapping with predictable, deterministic dataflow

Why Interleave Linear and Nonlinear Processing?

Distributed per-core

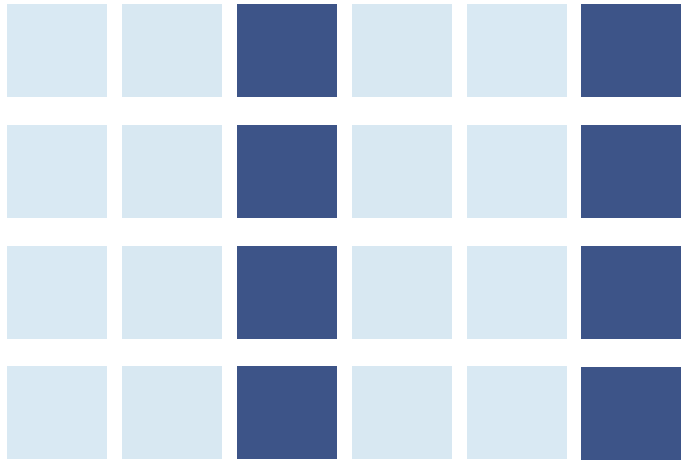


Local

High locality, high replication cost

Nonlinear slice

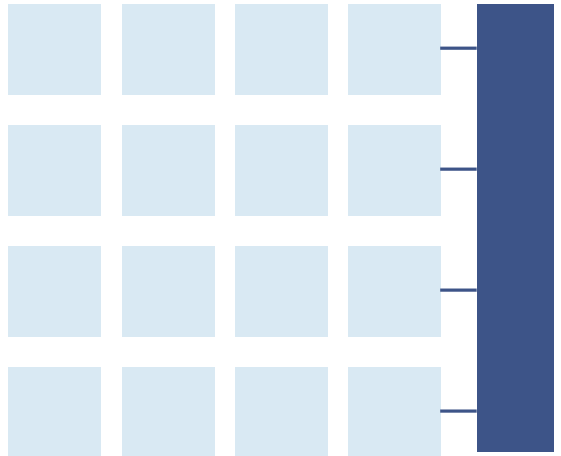
OmniAcc: interleaved slices



Interleaved

Localized sharing with regular placement

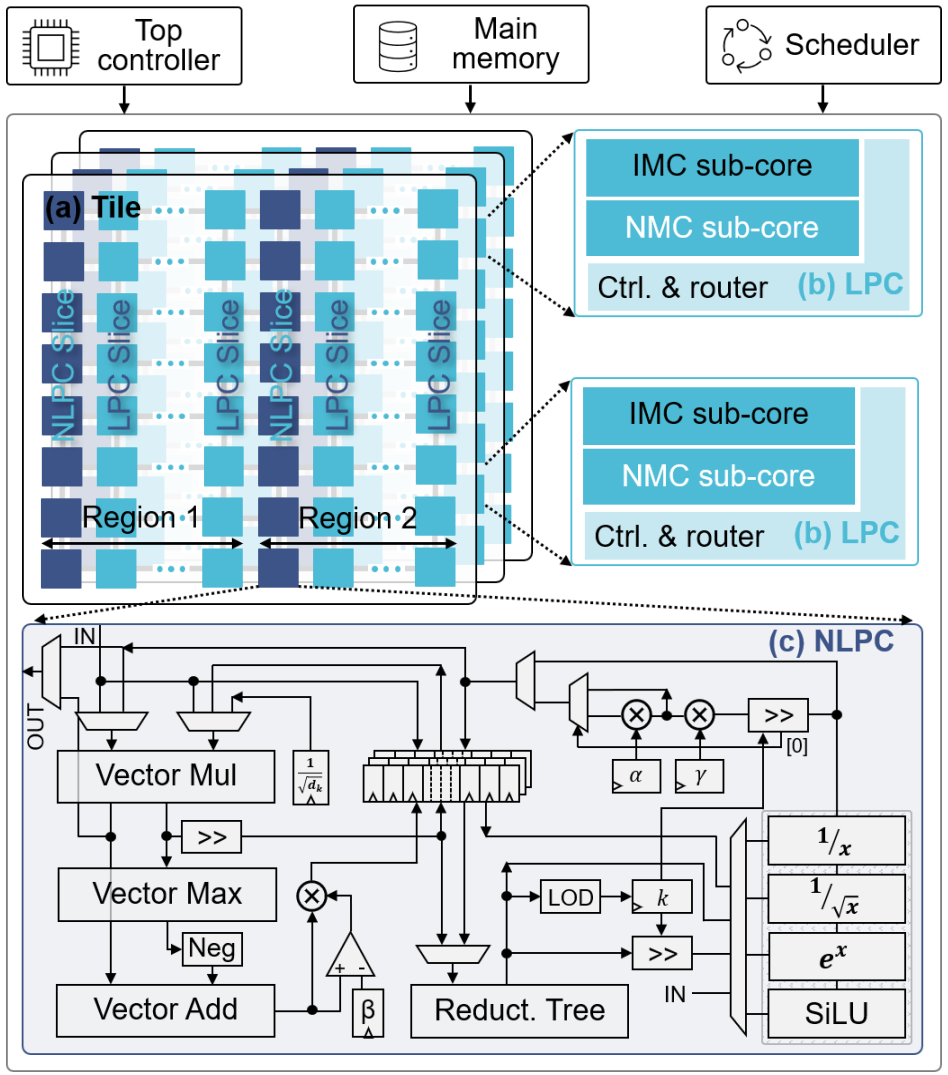
Central nonlinear units



Global

Reduced replication, longer communication

OmniAcc: Regularly Interleaved Compute Slices



Scalable processing tile array

- Column-wise functional slices
- Interleaved LPC & NLPC patterns

Linear processing core (LPC)

- In-memory computing (IMC) → matrix multiplication with stationary weights
- Near-memory computing (NMC) → dynamic-dynamic matrix multiplication

Nonlinear processing core (NLPC)

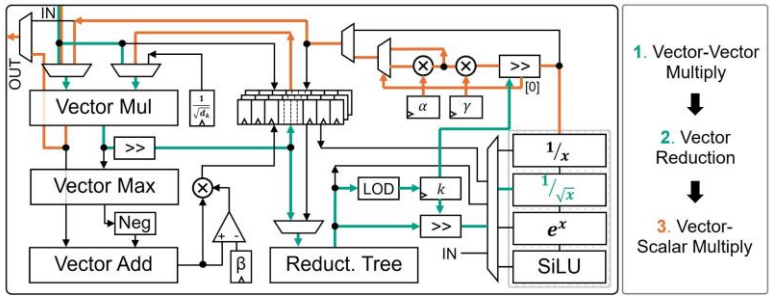
A local, reconfigurable fixed-point datapath supports RMSNorm, Softmax, and SiLU.

Router and local reduction

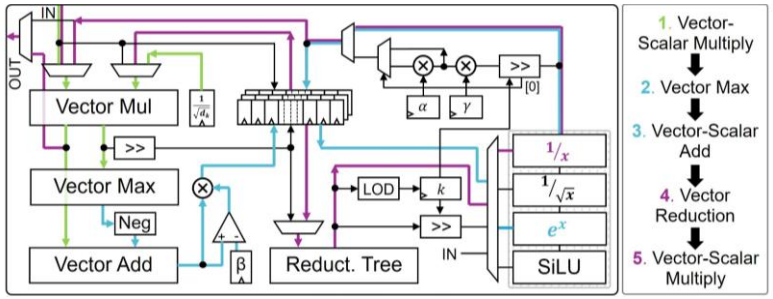
Router-integrated adders aggregate partial results

One Tile, Two Levels of Reconfiguration

Operation-level reconfiguration

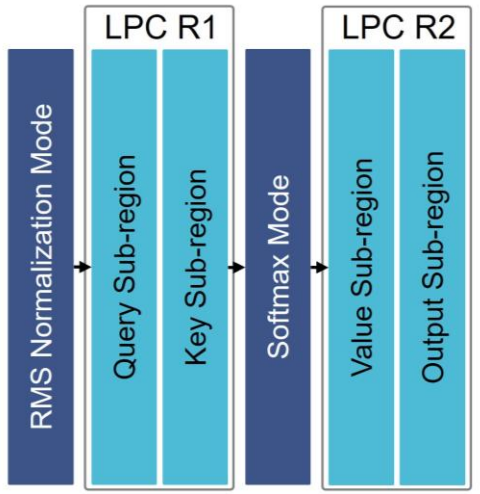


RMSNorm mode

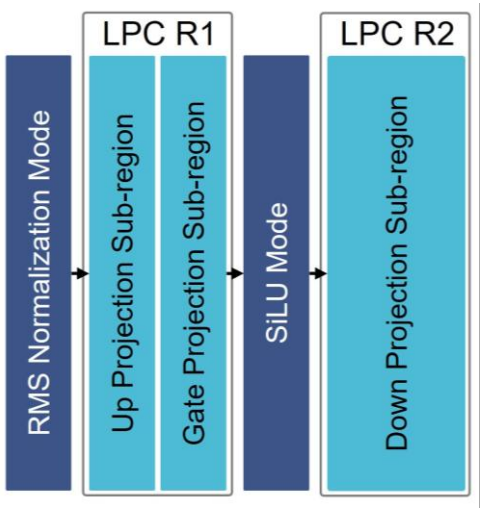


Softmax mode

Layer-level reconfiguration



Attention mode

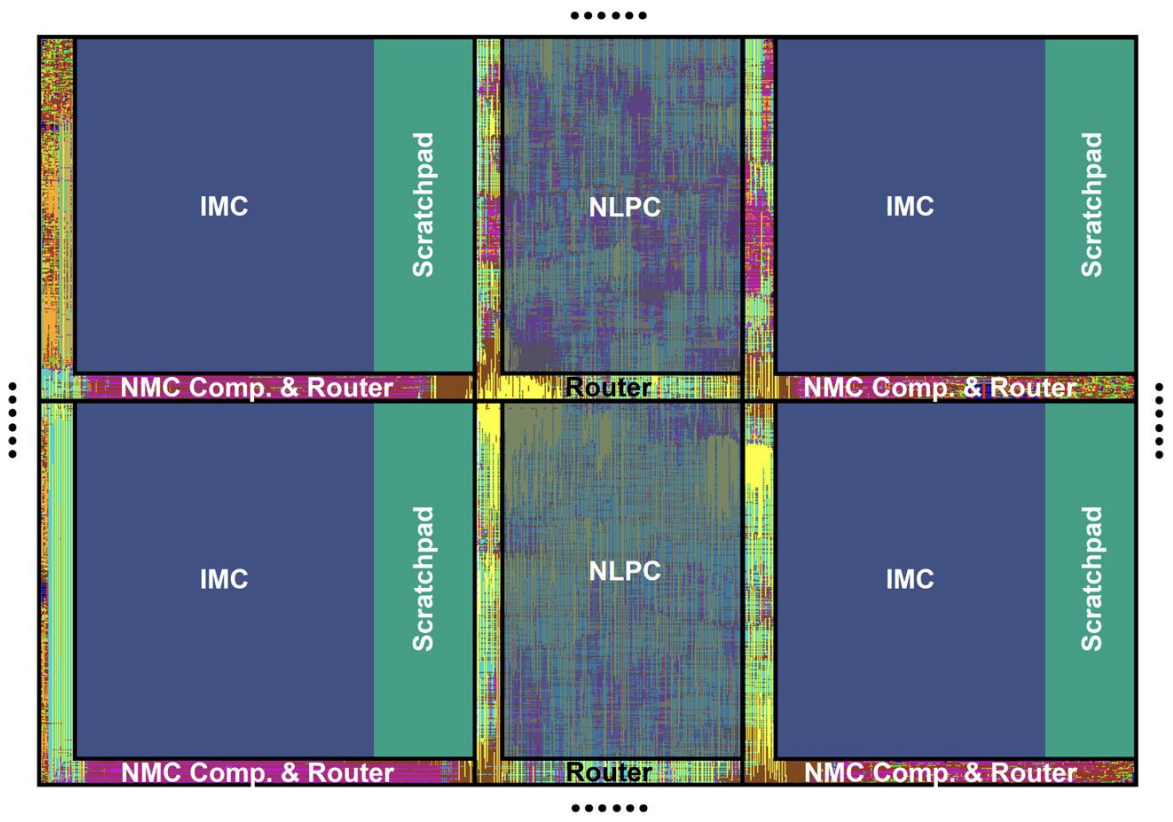


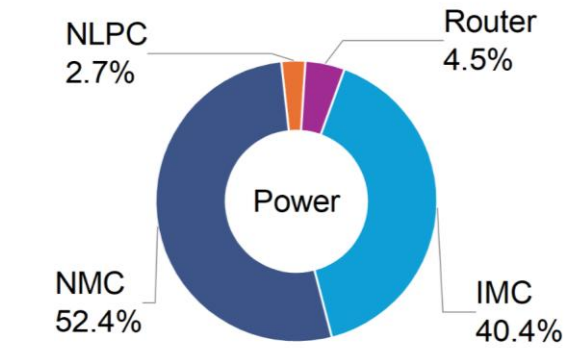
FFN mode

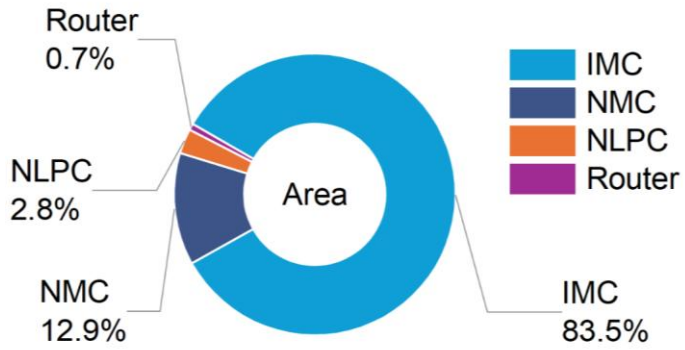
Shared NLPC executes RMSNorm, Softmax, or SiLU through selectable datapaths

The same tile supports attention and FFN by changing its functional assignments

Physical Implementation and Resource Breakdown

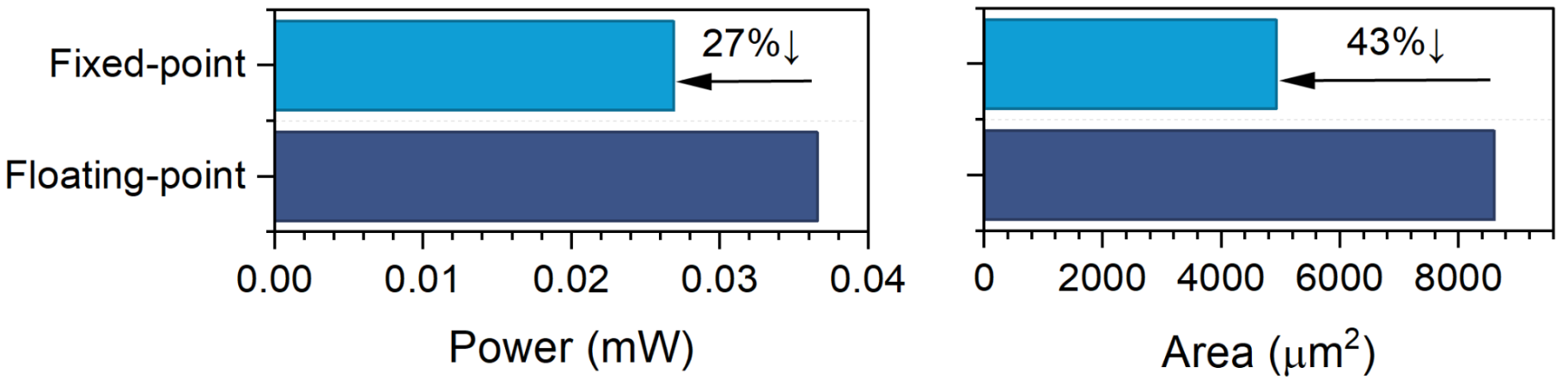






NLPC accounts for 2.8% of area and 2.7% of power

Fixed-Point NLPC Reduces Hardware Cost



27% lower power

43% lower area

Significant improvement compared with a floating-point NLPC implementation

Next: Locality-Aware Orchestration

RQ 1 | Can nonlinear functions use fixed-point arithmetic without material accuracy loss? — Algorithm

Solution 1 | Outlier-aware, accuracy-preserving fixed-point quantization

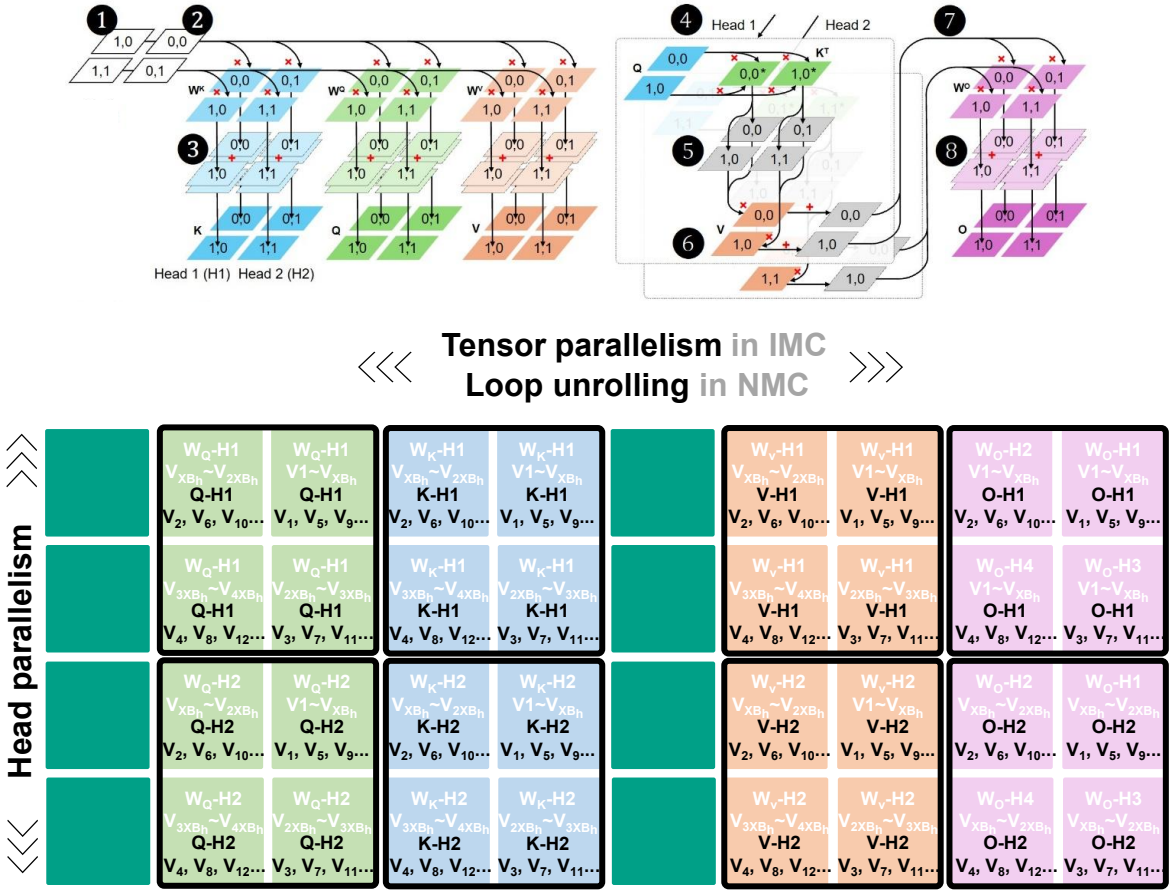
RQ 2 | How can nonlinear processing be integrated efficiently into a scalable architecture? — Architecture

Solution 2 | Reconfigurable nonlinear units and regularly interleaved functional slices

RQ 3 | How can computation and communication be orchestrated with low runtime overhead? — System

Solution 3 | Stationarity-aware mapping with predictable, deterministic dataflow

Locality-Aware Mapping and Fine-Grained Parallelism



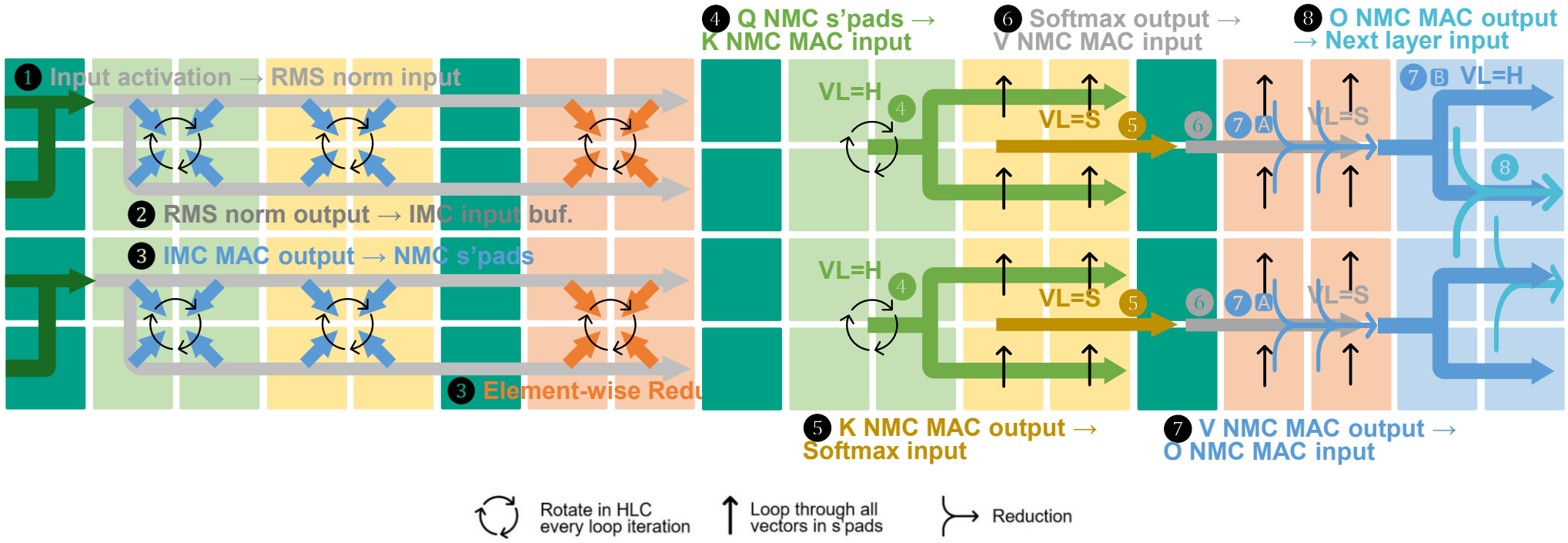
Head parallelism — Assign attention heads to local clusters

Tensor parallelism — Partition static weights across IMC arrays

Loop unrolling — Unroll NMC computation and distribute dynamic tokens cyclically

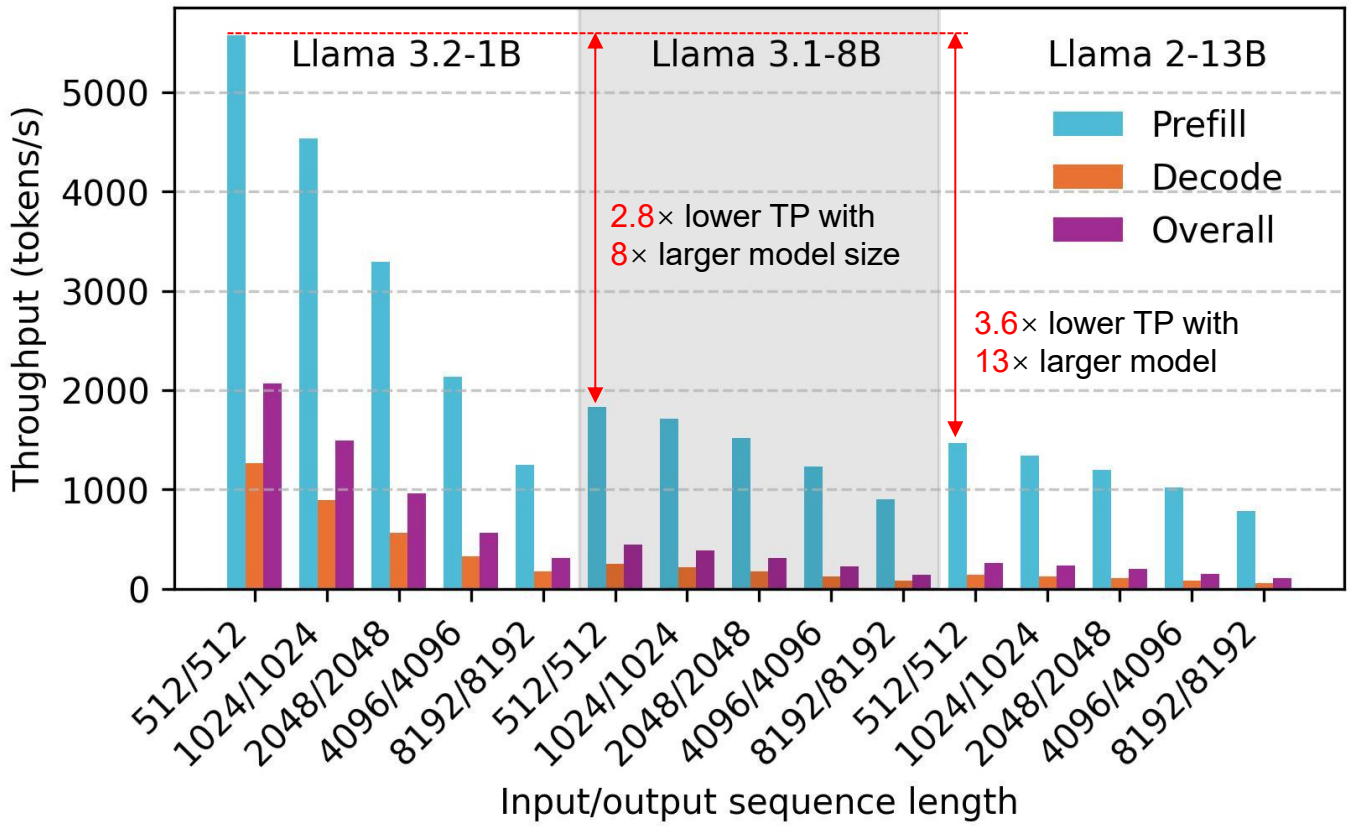
Map static weights to IMC, dynamic activations to NMC, and reductions near their producers

Deterministic Dataflow for Attention



Localized reductions and predetermined routes reduce dynamic control requirements

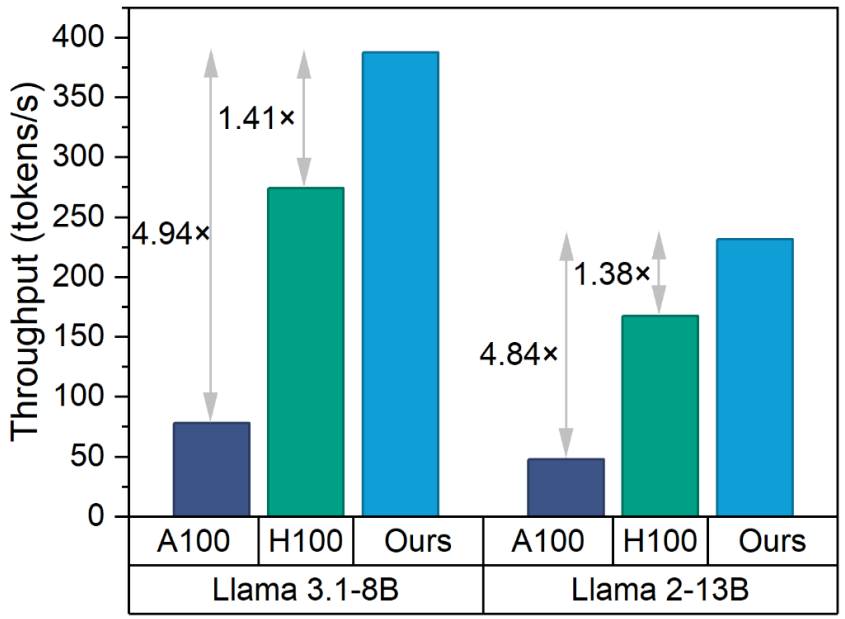
Throughput Scaling Across Models

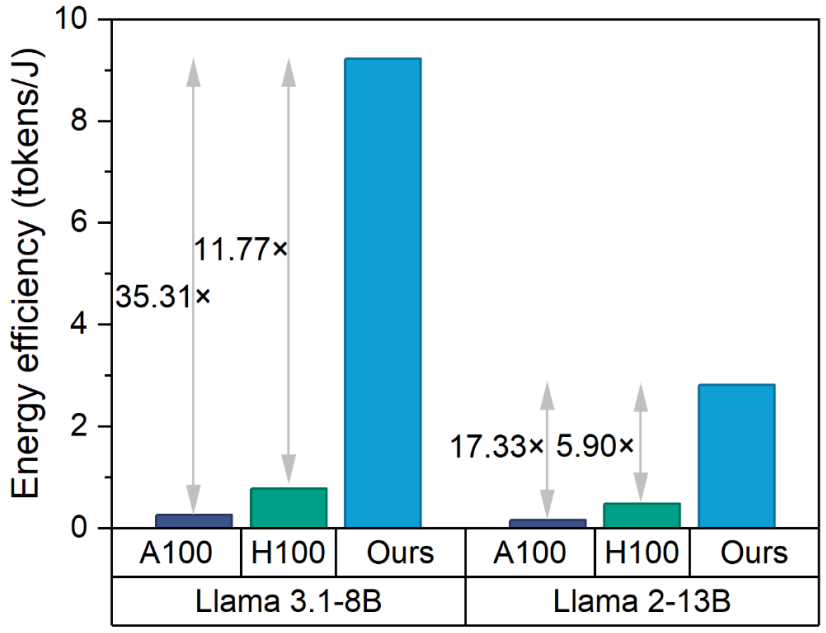


Prefill, decode, and overall throughput are evaluated for 1B, 8B, and 13B Llama models.

Throughput decreases less than proportionally as model size increases

System-Level Energy Efficiency Compared with GPUs





17.33–35.31× vs. A100

5.90–11.77× vs. H100

OmniAcc achieves substantial energy-efficiency gains while maintaining competitive throughput

Key Takeaways

- **Challenge:** Nonlinear operations incur nontrivial overhead; accelerating matrix multiplication alone does not guarantee efficient full-model inference
- **Circuit:** Outlier-aware, division-free nonlinear processing reduces NLPC power and area while preserving near-baseline model quality
- **Architecture:** Interleaved linear and nonlinear slices support attention and FFN on the same scalable tile
- **Dataflow:** Fine-grained parallelism and deterministic routing turn local compute into efficient full-model inference



Thank you!
Questions & Discussion